若依微服务项目部署流程

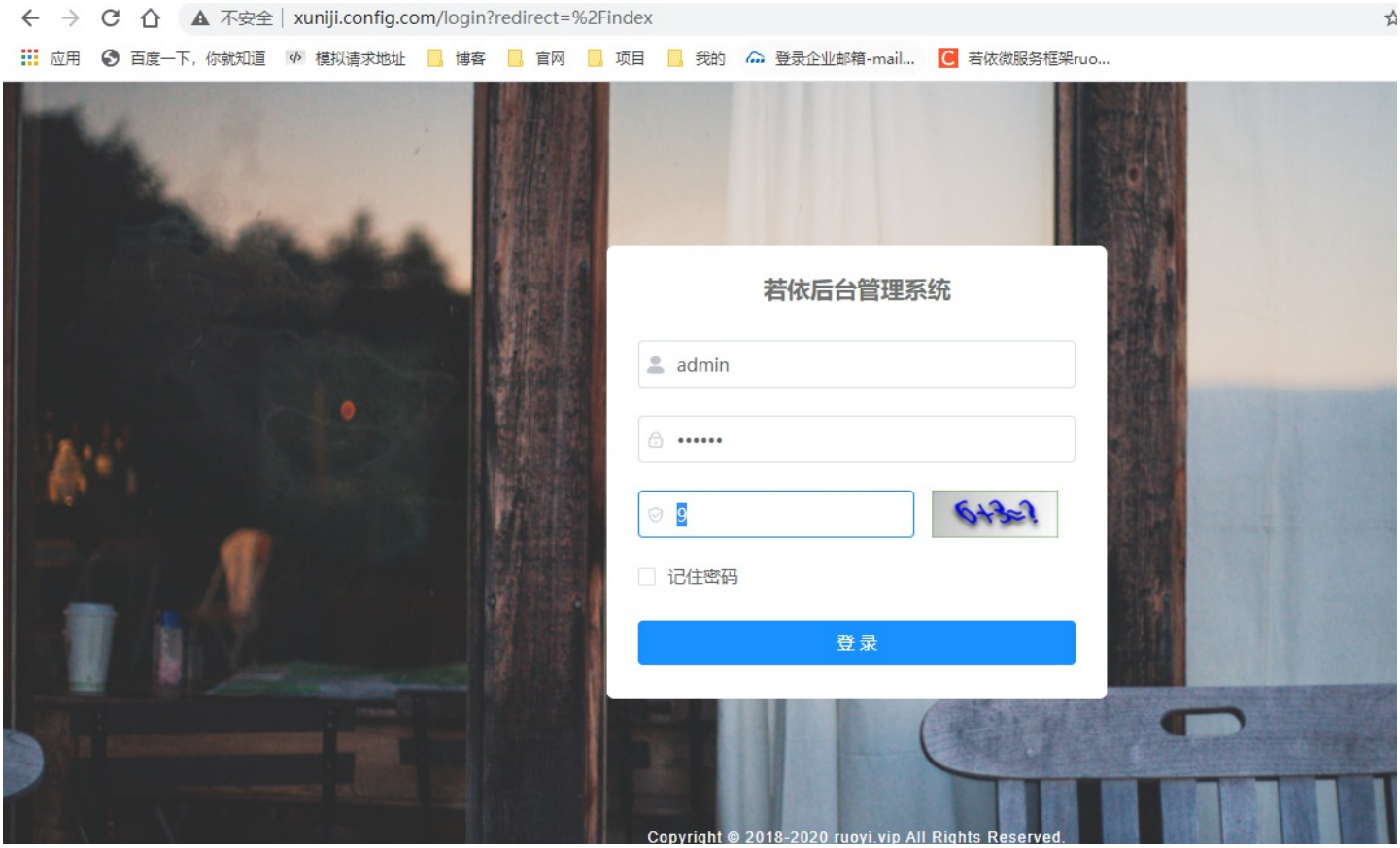
一.服务器说明

服务器一

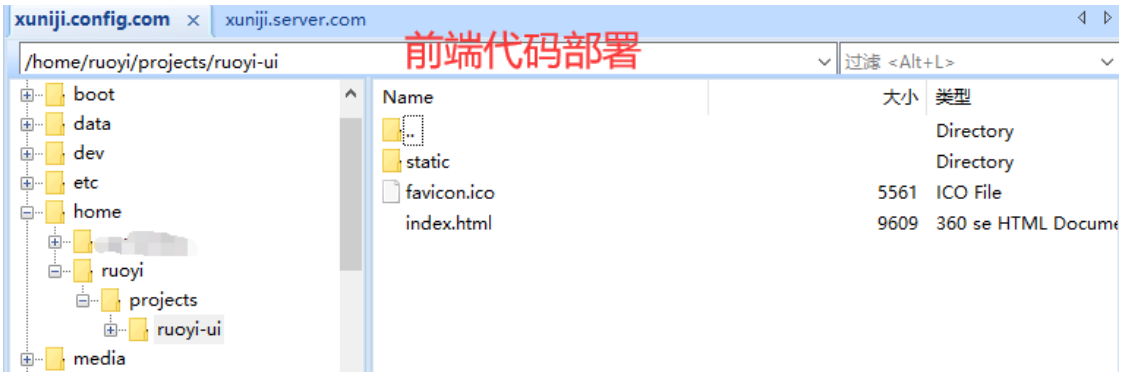
序号	域名	ip	作用
1	xuniji.config.com	192.168.220.128	提供redis,nacos,mysql ,nginx服务，前端代码也是部署到该服务器
2	xuniji.server.com	192.168.220.129	服务部署的机器，supervisor 进行项目线程统一管理

二.成品展示

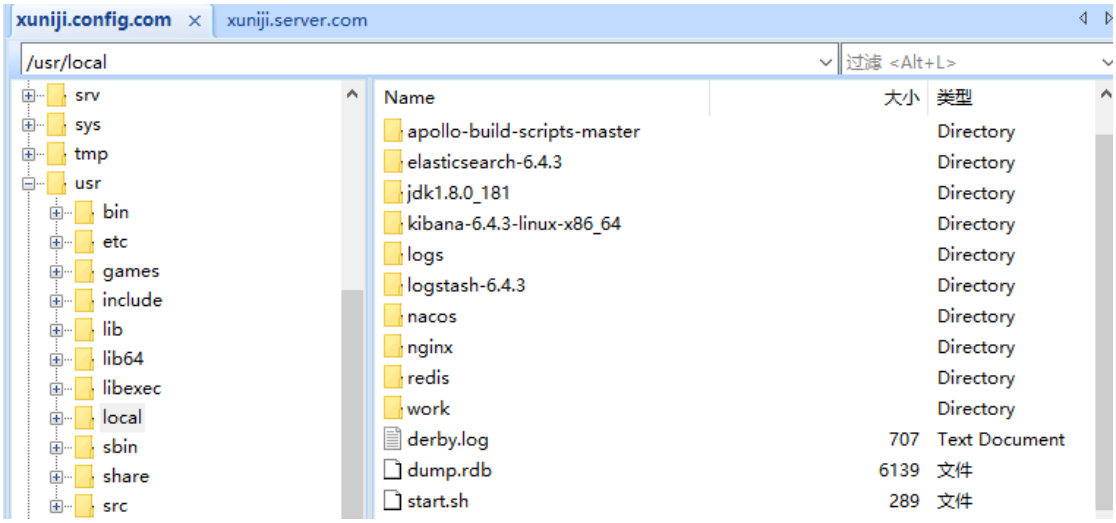
1.页面展示



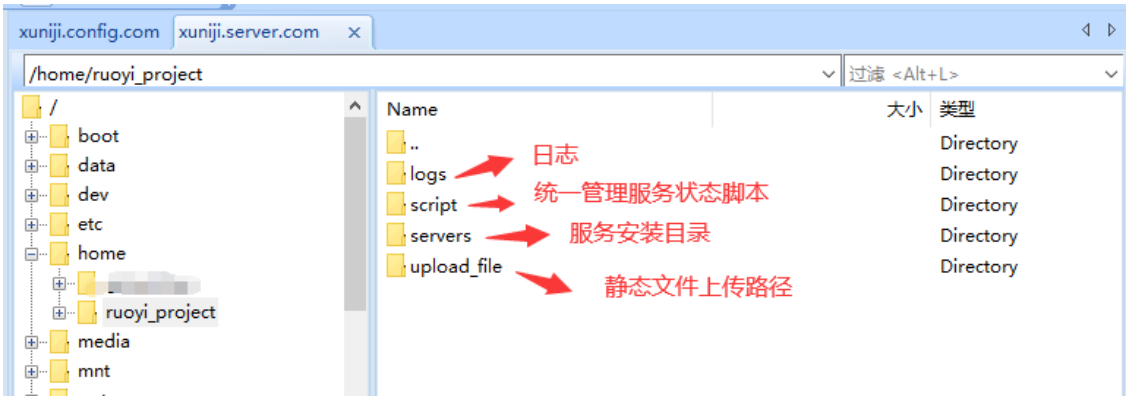
2.config机器前端代码部署路径

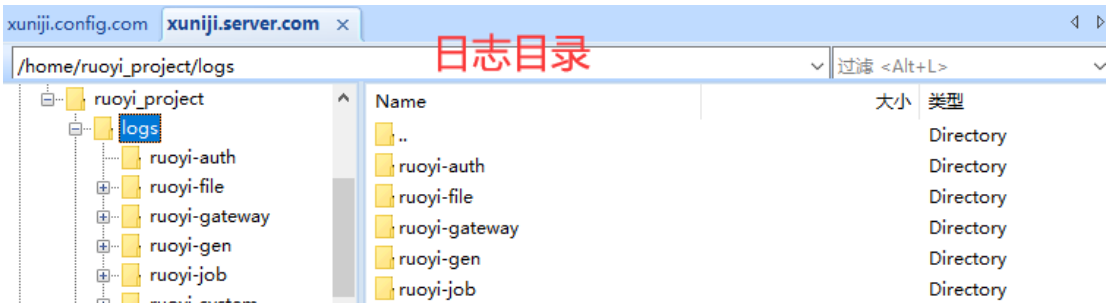


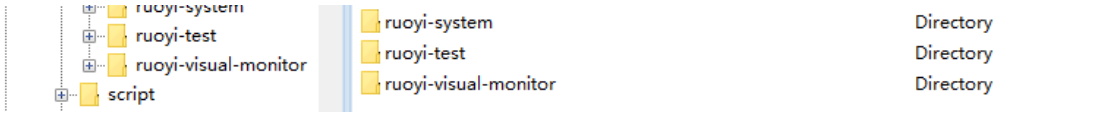
3.config机器中间件安装路径(只用到了redis，nginx，nacos其余的请忽略,start.sh是统一启动的shell脚本)

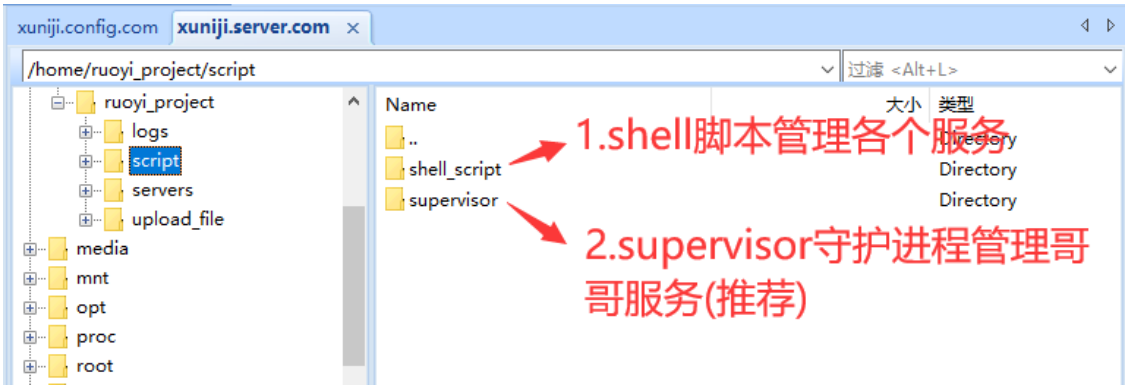


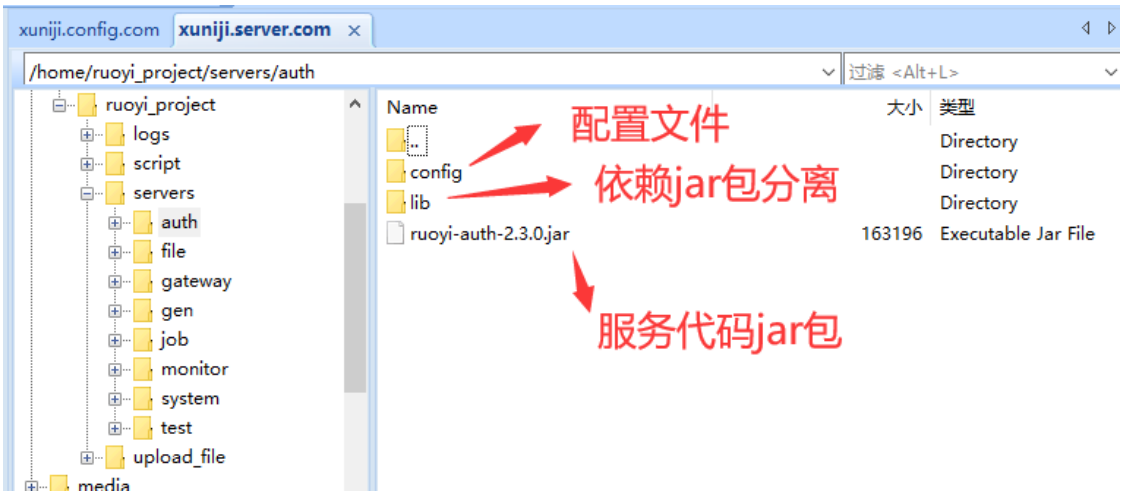
4.后端代码部署路径





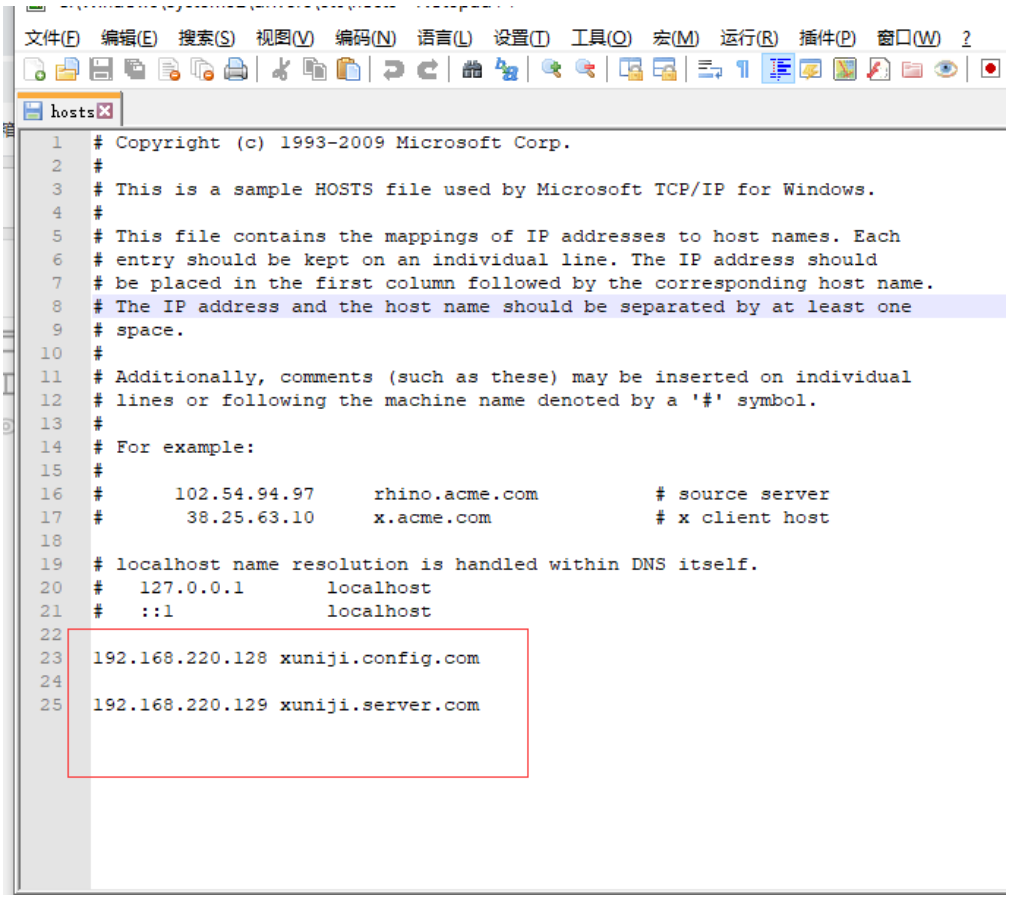

ruoyi-test
ruoyi-visual-monitor
script
ruoyi-system Directory
ruoyi-test Directory
ruoyi-visual-monitor Directory


xuniji.config.com
xuniji.server.com
/home/ruoyi_project/script
过滤 <Alt+L>
ruoyi_project
logs
script
servers
upload_file
media
mnt
opt
proc
root
Name
大小
类型
..
shell_script Directory
supervisor Directory
1.shell脚本管理各个服务
2.supervisor守护进程管理哥哥服务(推荐)


xuniji.config.com
xuniji.server.com
/home/ruoyi_project/servers/auth
过滤 <Alt+L>
ruoyi_project
logs
script
servers
auth
file
gateway
gen
job
monitor
system
test
upload_file
media
Name
大小
类型
..
config Directory
lib Directory
ruoyi-auth-2.3.0.jar 163196 Executable Jar File
配置文件
依赖jar包分离
服务代码jar包

三.前期准备

1.由于我是在虚拟机上部署的，并没有真正的域名所以需要修改本机的C:\Windows\System32\drivers\etc下的hosts文件



2.两台服务器的/etc下的hosts文件也需要修改

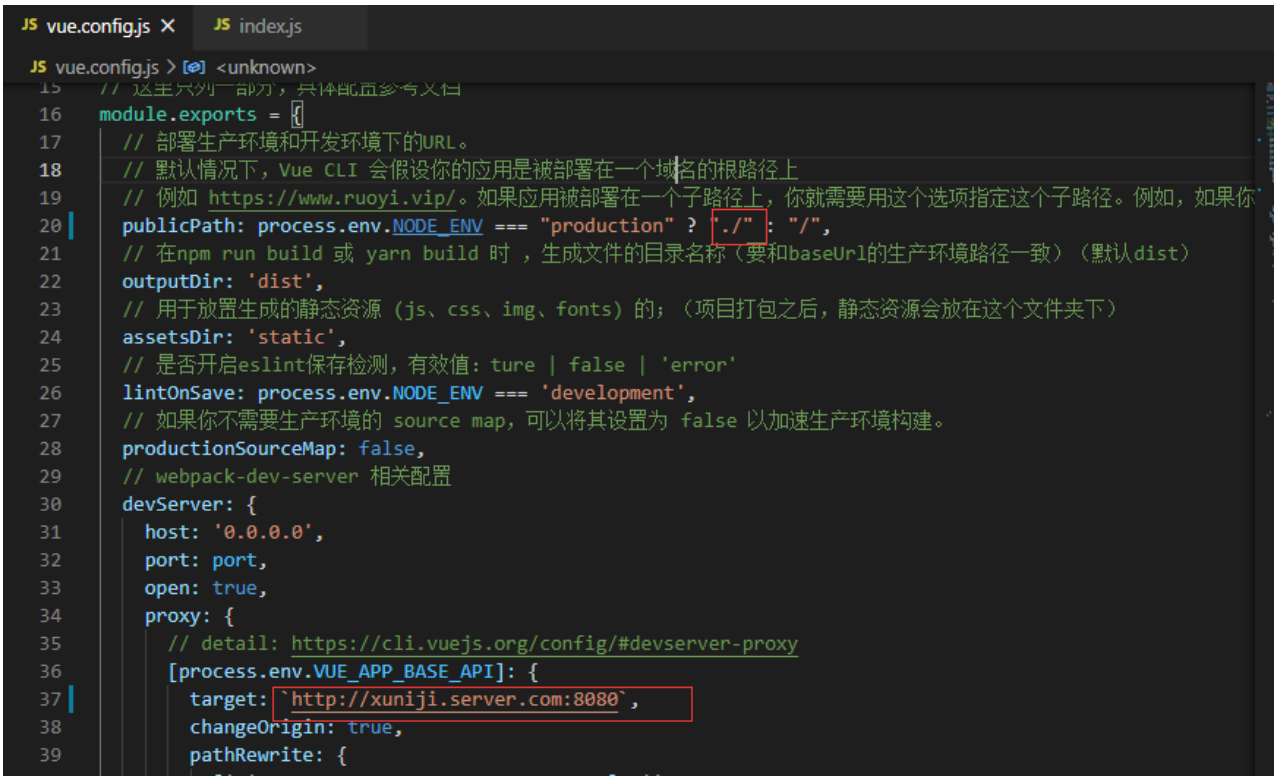
四.config机器前端部署

1.在config机器上安装jdk1.8,mysql，redis，nacos，nginx安装流程我就不一一解释了，大家可自行百度，后面我会贴上必要的配置信息

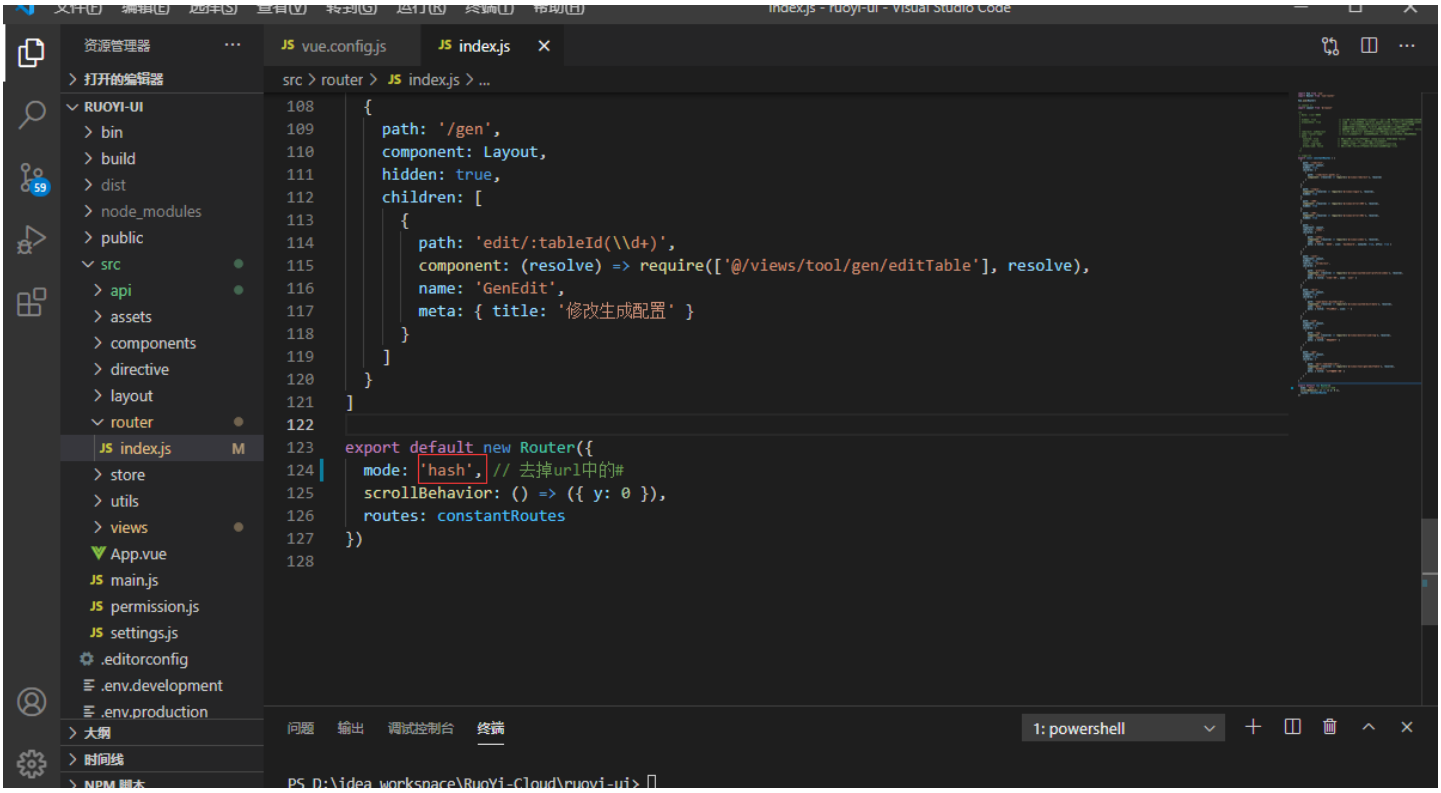
2.修改前端的vue.config.js的配置

配置修改：前端ui文件中的index.js文件、vue.config.js文件

如下图：



```
// 这里只列一部分，具体配置参考文档
module.exports = {
  // 部署生产环境和开发环境下的URL。
  // 默认情况下，Vue CLI 会假设你的应用是被部署在一个域名的根路径上
  // 例如 https://www.ruoyi.vip/。如果应用被部署在一个子路径上，你就需要用这个选项指定这个子路径。例如，如果你
  publicPath: process.env.NODE_ENV === "production" ? "./" : "/",
  // 在npm run build 或 yarn build 时 ，生成文件的目录名称（要和baseUrl的生产环境路径一致）（默认dist）
  outputDir: 'dist',
  // 用于放置生成的静态资源 (js、css、img、fonts) 的；（项目打包之后，静态资源会放在这个文件夹下）
  assetsDir: 'static',
  // 是否开启eslint保存检测，有效值: ture | false | 'error'
  lintOnSave: process.env.NODE_ENV === 'development',
  // 如果你不需要生产环境的 source map，可以将其设置为 false 以加速生产环境构建。
  productionSourceMap: false,
  // webpack-dev-server 相关配置
  devServer: {
    host: '0.0.0.0',
    port: port,
    open: true,
    proxy: {
      // detail: https://cli.vuejs.org/config/#devserver-proxy
      [process.env.VUE_APP_BASE_API]: {
        target: `http://xuniji.server.com:8080`,
        changeOrigin: true,
        pathRewrite: {
```



```
{
  path: '/gen',
  component: Layout,
  hidden: true,
  children: [
    {
      path: 'edit/:tableId(\\d+)',
      component: (resolve) => require(['@/views/tool/gen/editTable'], resolve),
      name: 'GenEdit',
      meta: { title: '修改生成配置' }
    }
  ]
}
]

export default new Router({
  mode: 'hash', // 去掉url中的#
  scrollBehavior: () => ({ y: 0 }),
  routes: constantRoutes
})
```

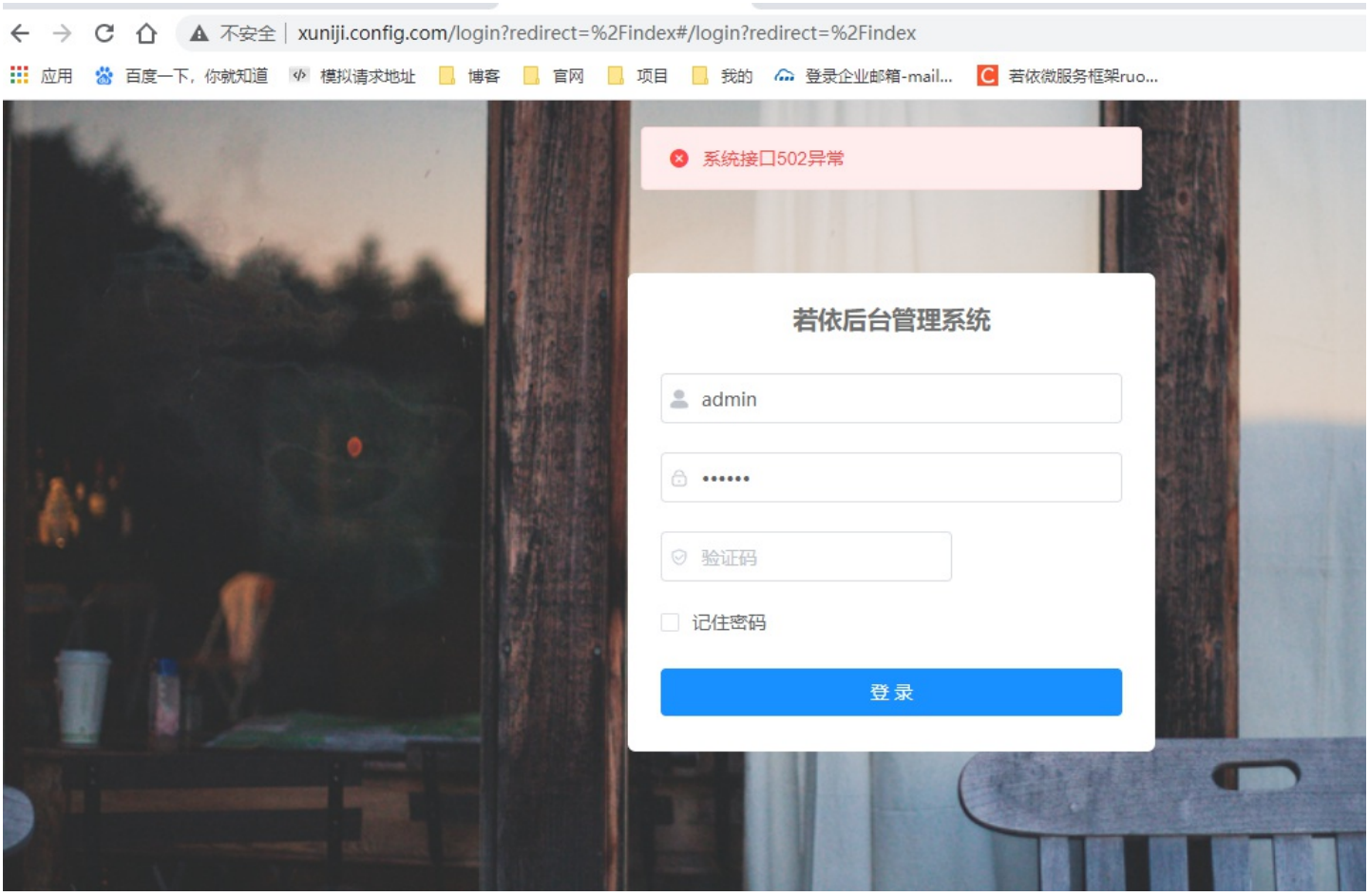
3.修改后在前端执行 npm run build:prod 生成dist包文件，放到config服务器的/home/ruoyi/projects/ruoyi-ui目录下

4.nginx.conf配置

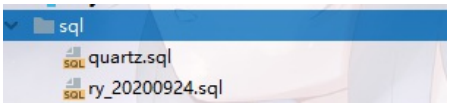
```
worker_processes  1;

events {
    worker_connections  1024;
}


http {
    include       mime.types;
    default_type  application/octet-stream;


    sendfile        on;

    keepalive_timeout  65;

    server {
        listen       80;
        server_name  localhost;

        location / {
            root   /home/ruoyi/projects/ruoyi-ui;
            try_files $uri $uri/ /index.html;
            index  index.html index.htm;
        }

        error_page   500 502 503 504  /50x.html;
        location = /50x.html {
            root   html;
        }

        location /prod-api/{
                proxy_set_header Host $http_host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header REMOTE-HOST $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_pass http://xuniji.server.com:8080/;
        }
    }
}
```

5.启动nginx

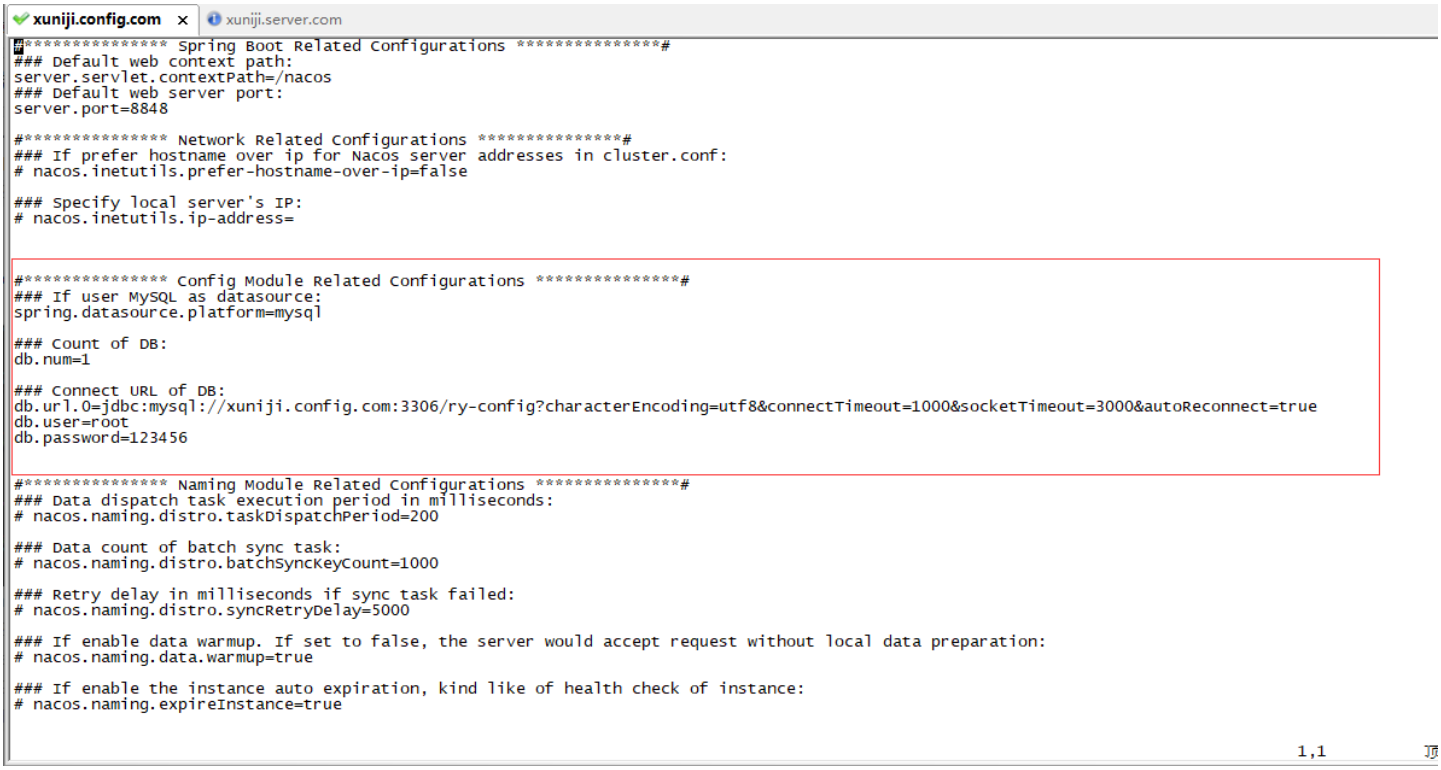
浏览器访问xuniji.config.com出现下图的效果则前端代码部署好了



6.将三个sql导入到mysql中



7.修改nacos的application.properties配置



8.统一启动redis，nacos,nginx的shell脚本

```
cd /usr/local
```

```
vim start.sh
```

```
#!/bin/bash

echo "启动redis"
/usr/local/redis/bin/redis-server /usr/local/redis/conf/redis.conf

echo "启动nacos"
/usr/local/nacos/bin/startup.sh -m standalone

echo "启动nginx"
/usr/local/nginx/sbin/nginx
```

```
chmod  u+x start.sh
```

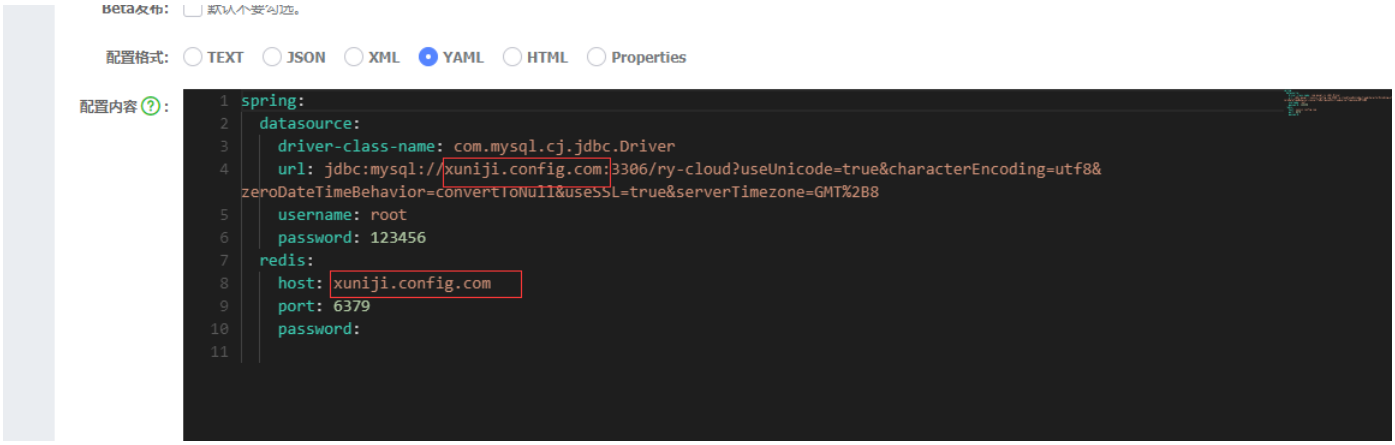
```
./start.sh
```

9.修改nacos配置信息

修改redis的ip

```yaml
spring:
  redis:
    host: xuniji.config.com
    port: 6379
```

修改mysql的ip



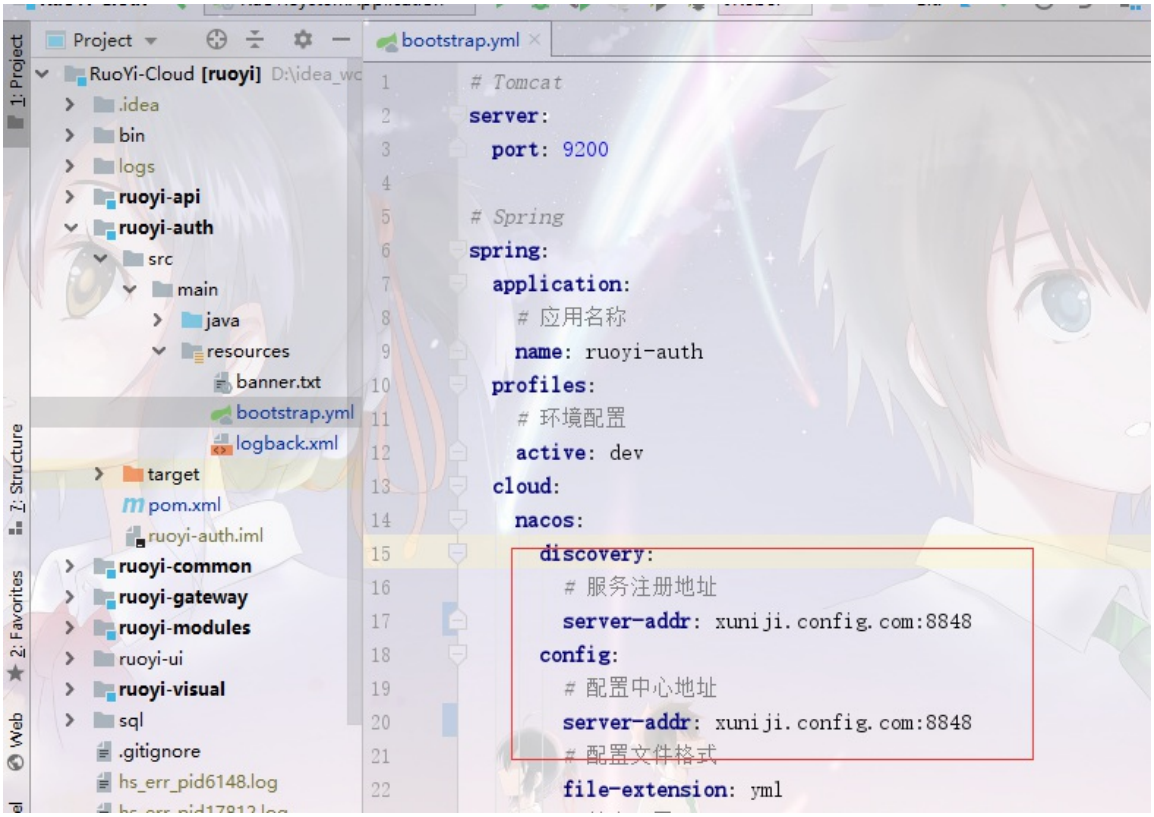
其余的服务也是一样，依次检查修改

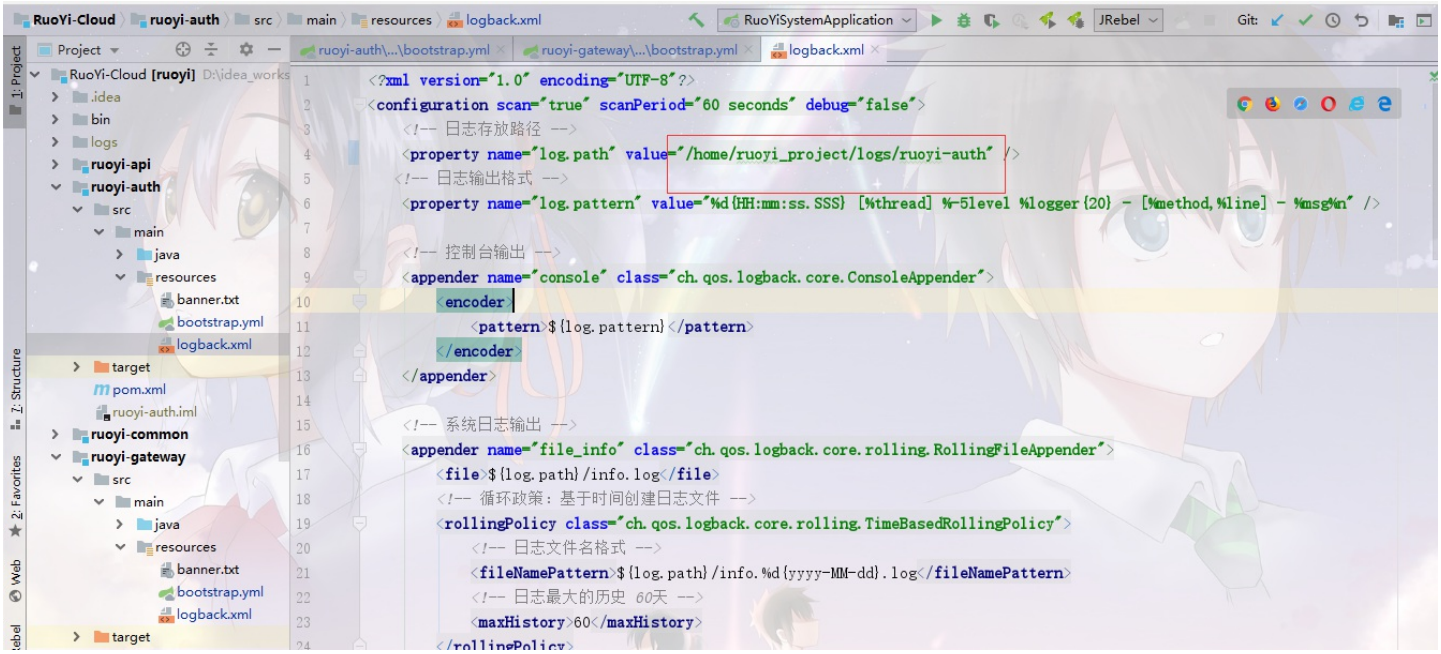
五.server机器后端部署

1.在server服务器上安装jdk1.8

2.修改项目的配置信息

以ruoyi-auth为例



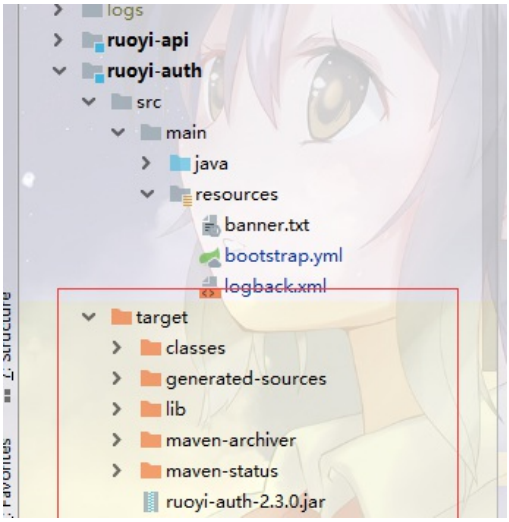


修改pom.xml文件，将build替换成下面的内容

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <executions>
                <execution>
                    <goals>
                        <goal>repackage</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
        <plugin>
            <!--打包时去除第三方依赖-->
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <layout>ZIP</layout>
                <includes>
                    <include>
                        <groupId>non-exists</groupId>
                        <artifactId>non-exists</artifactId>
                    </include>
                </includes>
            </configuration>
        </plugin>
<!--            拷贝第三方依赖文件到指定目录-->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-dependency-plugin</artifactId>
            <executions>
                <execution>
                    <id>copy-dependencies</id>
                    <phase>package</phase>
                    <goals>
                        <goal>copy-dependencies</goal>
                    </goals>
                    <configuration>
                        <!--target/lib是依赖jar包的输出目录，根据自己喜好配置-->
                        <outputDirectory>target/lib</outputDirectory>
                        <excludeTransitive>false</excludeTransitive>
                        <stripVersion>false</stripVersion>
                        <includeScope>runtime</includeScope>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

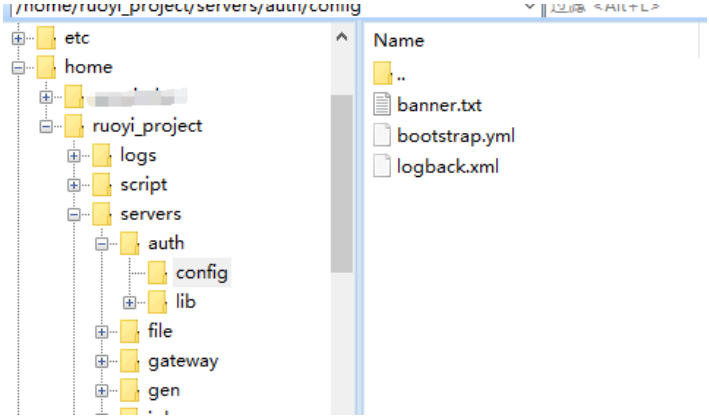
其余的服务也是一样，依次检查修改

3.idea打包项目

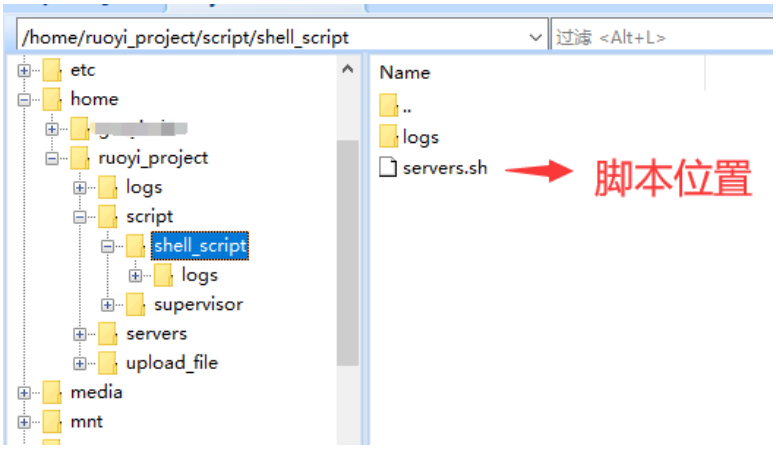


4.按照成品展示的第四步将文件夹创建好后开始上传jar包和依赖包，以及配置文件

配置文件我上传了这些



5.shell脚本管理jar包开启关闭(不推荐,shell脚本学习的话可以试一试)，test服务是我自己创建的一个服务，请忽略




```
#! /bin/bash
CURRENT_PATH="/home/ruoyi_project/servers/"
LOG_PATH="/home/ruoyi_project/script/shell_script/logs/"

SERVER_NAMES=("auth" "gateway" "file" "gen" "job" "system" "test" "monitor")
JAR=""
PID=""

#根据不同的服务定义项目路径和日志路径
```

```
if [[ "${SERVER_NAMES[@]}" =~ "${2}" ]]; then
    CURRENT_PATH=${CURRENT_PATH}${2}/
    LOG_PATH=${LOG_PATH}${2}.log
    JAR=$(find $CURRENT_PATH -maxdepth 1 -name "*.jar")
    PID=$(ps -ef | grep $JAR | grep -v grep | awk '{ print $2 }')
elif [[ "${2}" != "all" ]]; then
    echo -e "Usage: ./servers.sh ${1}  {auth|gateway|file|gen|job|system|test|monitor|all} \n" >&2
    exit 1
fi

#服务操作的通用方法
function call_servers(){
    for server_name in ${SERVER_NAMES[@]}
    do
        $1 $2 ${server_name}
    done
}

case "${1}" in
    "start")
        if [ "${2}" == "all" ];then
          call_servers ${0} ${1}
        elif [ ! -z "$PID" ]; then
          echo  "$JAR 已经启动，进程号: $PID "
        else
          echo -e "启动 $JAR ... \n"
          cd $CURRENT_PATH
        nohup java -Dloader.path=$CURRENT_PATH,resources,lib -Dfile.encoding=UTF-8 -jar $JAR >$LOG_PATH 2>&1 &
          if [ "$?"="0" ]; then
            echo -e "$JAR 启动完成，请查看日志确保成功 \n"
          else
            echo -e "$JAR 启动失败 \n"
          fi
        fi
        ;;
    "stop")
        if [ "${2}" == "all" ];then
          call_servers ${0} ${1}
        elif [ -z "$PID" ]; then
          echo -e "$JAR 没有在运行，无需关闭 \n"
        else
          echo -e "关闭 $JAR ..."
           kill -9 $PID
          if [ "$?"="0" ]; then
            echo -e "服务已关闭 \n"
          else
            echo -e "服务关闭失败 \n"
          fi
        fi
        ;;
    "restart")
        if [ "${2}" == "all" ];then
          call_servers ${0} ${1}
        else
          ${0} stop ${2}
          ${0} start ${2}
        fi
        ;;
    "kill")
        if [ "${2}" == "all" ];then
          call_servers ${0} ${1}
        else
          echo -e "强制关闭 $JAR \n"
          killall $JAR
          if [ "$?"="0" ]; then
```

```
            echo -e "成功 \n"
        else
            echo -e "失败 \n"
        fi
    fi
    ;;
  "status")
    if [ "${2}" == "all" ];then
      call_servers ${0} ${1}
    elif [ ! -z "$PID" ]; then
      echo -e "$JAR 正在运行 \n"
    else
      echo -e "$JAR 未在运行 \n"
    fi
    ;;
 *)
  echo -e "Usage: ./springboot.sh {start|stop|restart|status|kill} ${2} \n" >&2
    exit 1
esac
```

6.脚本效果展示

```
[root@xuniji shell_script]# ./servers.sh status auth
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

[root@xuniji shell_script]# ./servers.sh status all
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 未在运行

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 未在运行

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 未在运行

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 未在运行

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 未在运行

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 未在运行

[root@xuniji shell_script]# ./servers.sh start auth
启动 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...

/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 启动完成，请查看日志确保成功

[root@xuniji shell_script]# ./servers.sh stop auth
关闭 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...
服务已关闭

[root@xuniji shell_script]# ./servers.sh start all
启动 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...

/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar ...

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar ...

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar ...
```

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar ...

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar ...

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar ...

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar ...

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 启动完成，请查看日志确保成功

[root@xuniji shell_script]# *./servers.sh status all*
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 正在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 正在运行

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 正在运行

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 正在运行

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 正在运行

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 正在运行

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 正在运行

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 正在运行

[root@xuniji shell_script]# *./servers.sh stop all*
关闭 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar ...
服务已关闭

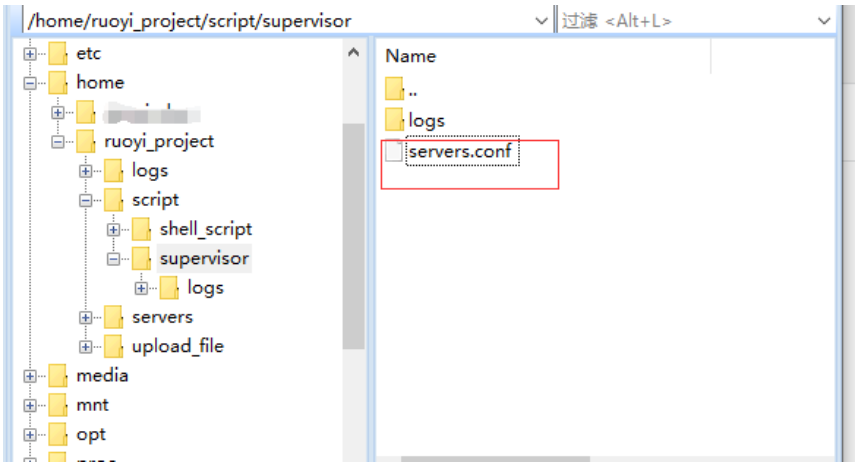
关闭 /home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar ...
服务已关闭

[root@xuniji shell_script]# *./servers.sh status all*
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 未在运行

```
/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 未在运行

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 未在运行

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 未在运行

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 未在运行

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 未在运行
```

7.通过supervisor来管理服务状态

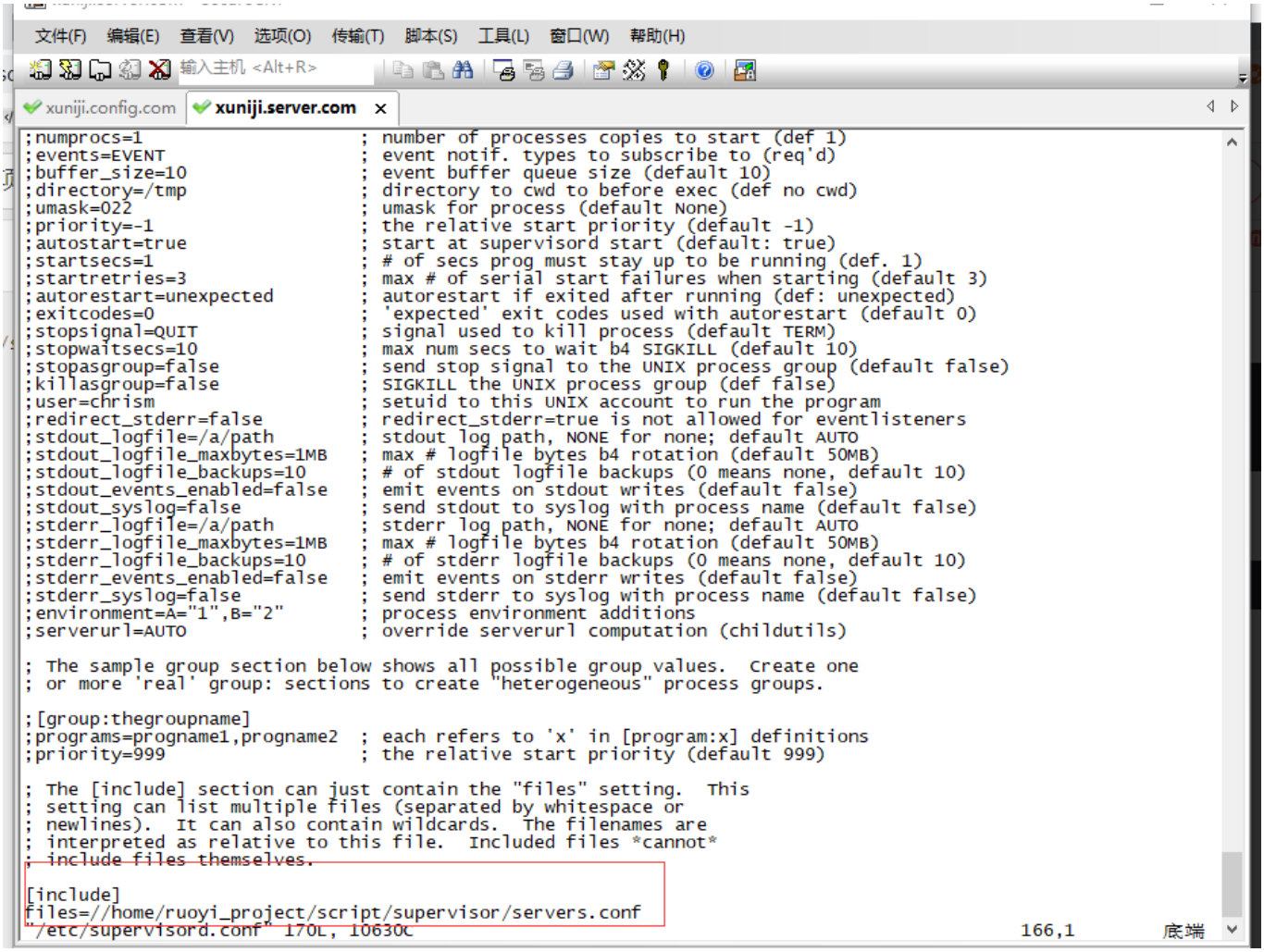


（1）安装守护程序supervisord(推荐)

```
yum install python-setuptools
easy_install supervisor
echo_supervisord_conf > /etc/supervisord.conf
```

（2）修改配置文件

```
vim /etc/supervisord.conf
```



(3)添加需要守护的程序到servers.conf

```
vim /home/ruoyi_project/script/supervisor/servers.conf
```

```
[program:auth]
;需要执行的命令
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/auth/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar
;命令执行的目录
directory=/home/ruoyi_project/servers/auth
;用户
user=root
stopsignal=INT
;是否自启动
autostart=true
;是否自动重启
autorestart=true
;自动重启时间间隔（s）
startsecs=3
;错误日志文件
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/auth_err.log
;输出日志文件
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/auth_info.log

[program:gateway]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/gateway/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar
directory=/home/ruoyi_project/servers/gateway
user=root
stopsignal=INT
autostart=true
```

```
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/gateway_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/gateway_info.log


[program:file]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/file/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar
directory=/home/ruoyi_project/servers/file
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/file_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/file_info.log

[program:gen]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/gen/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar
directory=/home/ruoyi_project/servers/gen
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/gen_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/gen_info.log


[program:job]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/job/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar
directory=/home/ruoyi_project/servers/job
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/job_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/job_info.log


[program:system]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/system/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar
directory=/home/ruoyi_project/servers/system
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/system_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/system_info.log

[program:monitor]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/monitor/lib,resources,lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar
directory=/home/ruoyi_project/servers/monitor
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
```

```
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/monitor_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/monitor_info.log
```

(4)设置supervisor开机启动,进入目录/usr/lib/systemd/system/新增文件supervisord.service，内容如下：

```
[Unit]
Description=Supervisor daemon
[Service]
Type=forking
ExecStart=/usr/bin/supervisord -c /etc/supervisord.conf
ExecStop=/usr/bin/supervisorctl shutdown
ExecReload=/usr/bin/supervisorctl reload
KillMode=process
Restart=on-failure
RestartSec=42s

[Install]
WantedBy=multi-user.target
```

执行

```
systemctl enable supervisord.service
```

8.supervisor效果展示

```
[root@xuniji script]# supervisorctl status
auth                             RUNNING   pid 33892, uptime 0:00:12
file                             RUNNING   pid 33283, uptime 0:04:16
gateway                          RUNNING   pid 33281, uptime 0:04:16
gen                              RUNNING   pid 33292, uptime 0:04:16
job                              RUNNING   pid 33282, uptime 0:04:16
monitor                          RUNNING   pid 33278, uptime 0:04:16
system                           RUNNING   pid 33279, uptime 0:04:16
test                             RUNNING   pid 33284, uptime 0:04:16
[root@xuniji script]# supervisorctl stop auth
auth: stopped
[root@xuniji script]# supervisorctl start auth
auth: started
[root@xuniji script]# supervisorctl stop all
monitor: stopped
auth: stopped
gateway: stopped
test: stopped
gen: stopped
job: stopped
file: stopped
system: stopped
```

supervisorctl stop all 停止全部
supervisorctl stop program_name # 停止某一个进程，program_name 为 [program:x] 里的x

supervisorctl start program_name # 启动个进程
supervisorctl restart program_name # 重启某个进程
supervisorctl update 修改配置文件后更新配置

END(以上只是个人的一些部署想法和方式，如果大家有更好的方式，欢迎提出来，互相学习)